SYNOPSIS

ON

Load Balancer with Health Checks

Submitted By:

Sonali Patel (2315002185)

Sumit Kaushik (2315002231)

Tushant Pratap Singh (2315002319)

Uday Dixit (2315002333)

Udit Raghav (2315002338)

Submitted To:

Mr. Ayush Kumar

TITLE OF THE PROJECT

Load Balancer with Health Checks

OBJECTIVE

- In modern web applications, handling all user requests with a single backend server can lead to performance bottlenecks, system failures, and poor user experience during high traffic or server crashes. There is a need for a reliable mechanism to distribute traffic and ensure continuous service availability.

- This project aims to design and implement a Node.js-based load balancer using an Express gateway that distributes incoming client requests across multiple backend servers. The system continuously monitors backend health, automatically removes unhealthy servers, and maintains backend status and configuration using MongoDB.

- The main objective is to improve system reliability, fault tolerance, and scalability by intelligently routing traffic using round-robin scheduling with a least-connections fallback, ensuring that user requests are always served by healthy and available backend servers.

SCOPE

This project demonstrates how incoming client traffic can be efficiently managed and distributed across multiple backend servers to ensure high availability, reliability, and fault tolerance in web applications. It showcases the implementation of a custom load balancer with health monitoring, intelligent request routing, and automated backend management using Node.js–based technologies.

What the Project Will Cover

- Design and implementation of an Express.js gateway acting as a centralized load balancer.
- Distribution of client requests across multiple backend Node.js servers.
- Asynchronous health checking of backend servers and automatic removal of unhealthy nodes.
- Storage and tracking of backend status, configuration, and health history using MongoDB.
- Implementation of round-robin load balancing with least-connections fallback for high traffic scenarios.
- Automation of backend deployment and management using shell scripts.
- Version control of backend configuration changes using Git.
- Efficient communication between gateway and backends using Unix TCP sockets.

FEATURES

1. **Request Load Balancing:** Distributes incoming client requests across multiple backend servers to avoid overload.
2. **Health Monitoring:** Continuously checks backend server health and removes failed servers automatically.
3. **Intelligent Routing:** Uses round-robin scheduling with least-connections fallback during traffic spikes.
4. **Backend State Management:** Stores server status and health history in MongoDB for persistence and analysis.
5. **Automation Support:** Uses shell scripts to deploy, add, or remove backend servers efficiently.
6. **Configuration Versioning:** Tracks backend configuration changes using Git for easy rollback and auditing.
7. **Optimized Communication:** Uses Unix TCP sockets for faster and efficient request forwarding.

PROPOSED SYSTEM

The proposed system implements a custom load balancer that efficiently distributes incoming client requests across multiple backend servers while continuously monitoring their health. The system ensures high availability and fault tolerance by routing traffic only to healthy and available backend servers.

Workflow Steps:

1. Client sends a request to the Express-based load balancer (gateway).
2. The load balancer queries MongoDB to identify healthy backend servers.
3. A backend server is selected using the round-robin algorithm, with least-connections used as a fallback during high traffic.
4. The request is forwarded to the selected backend server using Unix TCP socket communication.
5. The backend server processes the request and sends the response back to the load balancer.
6. The load balancer returns the response to the client.
7. In parallel, a health monitoring module periodically checks backend servers and updates their status in MongoDB.
8. Unhealthy backend servers are automatically removed from the active pool, ensuring uninterrupted service.

This approach guarantees that user requests are handled efficiently, system failures are managed automatically, and the application remains reliable and scalable under varying traffic conditions.

TOOLS & TECHNOLOGIES

Tool / Technology	Purpose
Node.js	Backend runtime environment for gateway and servers
Express.js	Implementation of the load balancer gateway
MongoDB	Storage of backend status, configuration, and health history
Shell Scripting (Bash)	Automation of backend deployment and management
Git	Version control for backend configurations
GitHub	Repository management and collaboration
Unix TCP Sockets	Efficient communication between gateway and backend servers
HTTP/REST	Request forwarding and health check communication

HARDWARE & SOFTWARE REQUIREMENTS

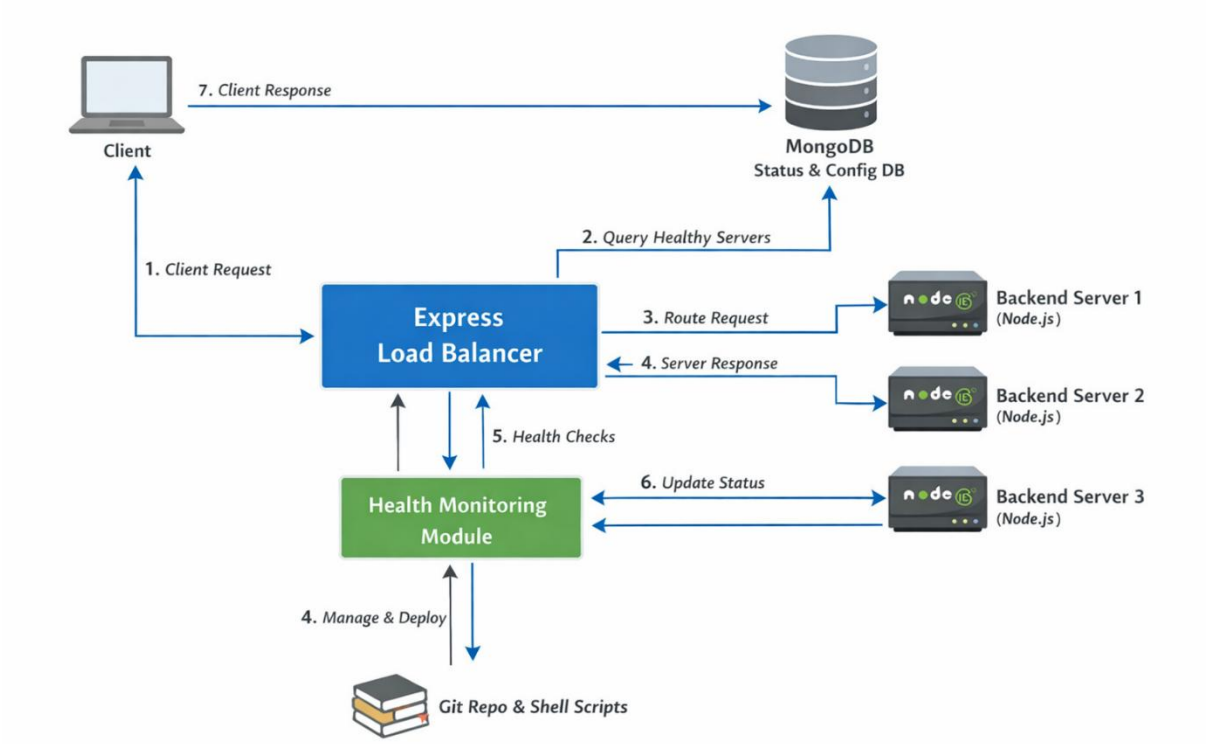
Hardware

- Laptop/PC with minimum **8 GB RAM**, **Intel i5 processor**
- Stable internet connection

Software

- **Node.js:** (v18 or later)
- **Express.js:** Web framework for gateway implementation
- **MongoDB:** Database for backend status and health tracking
- **Git & GitHub:** Version control and repository management
- **Shell Environment:** Bash / PowerShell for automation scripts

Architecture



EXPECTED OUTCOME

By the end of the project:

- A fully functional load balancer capable of distributing client requests across multiple backend servers will be implemented.
- The system will automatically monitor backend server health and route traffic only to healthy and available servers.
- Server failures and traffic overloads will be handled without interrupting user requests, ensuring high availability.
- Developers will gain practical experience in backend scalability, fault tolerance, and real-world system design concepts.

APPLICATIONS & USES

- Suitable for academic and professional projects demonstrating load balancing and backend system design
- Can be extended for use in production web applications requiring high availability and fault tolerance
- Provides a foundational model for reverse proxies and traffic management systems such as NGINX and HAProxy.
- Serves as a learning platform for understanding scalability, reliability, and DevOps-oriented backend architectures.

CONCLUSION

This project demonstrates the design and implementation of a custom load balancer that efficiently distributes client requests across multiple backend servers while continuously monitoring their health. By automating health checks, intelligent traffic routing, and backend management, the system ensures high availability, reliability, and fault tolerance. The implementation reflects real-world load balancing concepts used in production systems and provides practical exposure to backend scalability, system reliability, and infrastructure-level application design.